Guided Final Projects in AI for Fluid and Kinetic Flows

Weeks 5-6 Project Guide and Notebook Map (Expanded self-study edition v1.4.2)

MIE 690A - AI in Fluid Mechanics

Summer 2026

Central message

You are not being asked to invent an entire AI-CFD project from nothing. You will begin from a working Week-4 baseline, make one scientifically meaningful extension, and use controlled numerical and physical tests to determine whether that extension is useful.

Instructor: Ehsan Roohi

Contents

1 Executive Summary

Week 4 taught a common workflow on the lid-driven cavity:

fixed CFD solver → quality-checked fields → baseline surrogate → field-aware model → validation.

Weeks 5 and 6 use this common foundation to teach a new skill: *how to design and defend a controlled computational experiment*. Every student selects one of eleven bounded project variants. Track 6 is an instructor-approved research track with substantially more supplied code because it deploys a neural closure inside a particle Fokker–Planck solver. Starter notebooks provide the data loader, baseline model, metrics, and plotting functions. The student owns the split, the one permitted modification, the controlled comparison, the physical interpretation, and the failure analysis.

Important boundary

Simply rerunning or cosmetically editing a Week-4 notebook is not a final project. Conversely, students are not expected to rewrite the CFD solver, design a research codebase from scratch, or build an unrelated project using automatically generated code.

1.1 What is new relative to Week 4?

	Week 4	**Weeks 5-6**
Question	Supplied by the instructor	Selected from one bounded project variant
Code	Guided laboratory notebook	Working starter plus designated experiment choices
Model	Reproduce and compare supplied baselines	Change one scientifically meaningful element
Validation	Follow required metrics	Predeclare criteria, analyze tradeoffs, find failure
Conclusion	Explain notebook output	Defend a claim supported by controlled evidence

2 Learning Objectives

By the end of the project, a student should be able to:

1. turn a broad idea into a one-sentence research question;
2. distinguish training, validation, blind test, interpolation, and extrapolation;
3. change one factor at a time and preserve a defensible baseline;
4. evaluate a fluid surrogate with numerical and physical evidence;
5. separate model error from data quality, stochastic variation, and numerical bias;
6. identify a condition where the model is unreliable;
7. explain and modify the important cells in the submitted notebook;
8. document any AI-assisted coding and verify the resulting implementation.

3 How to Use This Guide

This document is written as a self-study project manual, not only as a list of deliverables. Each track is organized in four layers:

1. **Concept layer:** the physical and machine-learning idea is explained before the code is discussed.
2. **Provided-code layer:** the notebook cells and helper functions that are already complete are identified.
3. **Student-decision layer:** the small number of choices that the student is allowed to change are stated explicitly.
4. **Evidence layer:** the tables, figures, physical checks, and failure test required to support a conclusion are listed.

Students should first run `P0_Project_Setup.ipynb`, then read the introductory pages for the selected track, and only then edit the track notebook. The project is intentionally guided: the solver, data loader, most plotting functions, and baseline model are supplied. The independent work is the scientific design, the controlled modification, and the interpretation.

Central message

A student is not expected to understand every helper function before beginning. The student *is* expected to understand the variables being changed, the meaning of the train/validation/test split, the metric used for selection, and the physical meaning of the final plots.

4 Essential Notation and Definitions

The same symbols are used throughout the guide and notebooks. This section defines them once so that later equations can be read without guessing.

4.1 Physical variables for the cavity problem

Symbol	Name	Meaning in this course
L	cavity length	Characteristic side length of the square cavity. Coordinates are normalized by L, so the computational domain is $0 \le x/L \le 1$, $0 \le y/L \le 1$.
U_{lid}	lid speed	Tangential speed of the moving top wall. Velocity is often normalized by this value.
ν	kinematic viscosity	Momentum diffusivity of the continuum fluid.
Re	Reynolds number	$Re = U_{\text{lid}} L/\nu$. It measures the ratio of inertial to viscous effects. Increasing Re generally sharpens gradients and changes the vortex structure.
x, y	spatial coordinates	Nondimensional coordinates in the cavity. In code, arrays named `x` and `y` contain the grid coordinates.
$u(x,y), v(x,y)$	velocity components	Horizontal and vertical velocity components. The moving lid mainly drives positive u near the top wall.
$p(x,y)$	gauge pressure	Pressure recovered from the velocity field and shifted so that its spatial mean is zero. Only pressure differences are physically meaningful in this formulation.
$\psi(x,y)$	streamfunction	A scalar field used in the Week-4 solver. With the convention used in the notebooks, $u = \partial\psi/\partial y$ and $v = -\partial\psi/\partial x$. This automatically gives a divergence-free discrete reference velocity field.
$\omega(x,y)$	vorticity	In two dimensions, $\omega = \partial v/\partial x - \partial u/\partial y$. It measures local rotation of the flow.
λ	mean free path	Average molecular distance between collisions in the kinetic/rarefied track.
Kn	Knudsen number	$Kn = \lambda/L$. It measures rarefaction. Small Kn is continuum-like; larger Kn requires kinetic descriptions.

4.2 Grid, field, and case notation

A **grid point** is one spatial location (x_j, y_j). A **field** is the value of a variable at every grid point, for example the full u array at one Reynolds number. A **case** is one complete physical simulation condition, such as one full cavity solution at $Re = 275$ or one stochastic kinetic run at a specified $(Kn, U_{\text{wall}}, \text{seed})$.

Let N_x and N_y be the number of grid points in the two directions and let

$$N_g = N_x N_y$$

be the number of spatial points in one scalar field. In the supplied continuum dataset, $N_x = N_y = 65$, so $N_g = 4225$. A three-output field (u, v, p) therefore contains $3N_g = 12675$ scalar values per physical case.

Important boundary

Grid points from the same case are not independent physical simulations. Randomly sending some points from one case to training and other points from the same case to testing creates leakage. The project notebooks split complete cases, not random points.

4.3 Machine-learning notation

Symbol	Name	Meaning
$\mathbf{z}$	model input	Input vector supplied to a network. For a coordinate DNN, $\mathbf{z} = (Re, x, y)$. For Track 5, $\mathbf{z} = (\log_{10} Kn, U_{\text{wall}}, x, y)$.
$\mathbf{y}$	target/label	Reference output used for training, for example (u, v, p). The label may contain numerical error or stochastic noise.
$\widehat{\mathbf{y}}$	prediction	Model output. A hat always denotes a predicted or reconstructed quantity.
f_θ	neural model	Function represented by a neural network. θ denotes all trainable weights and biases.
$\mathcal{L}_{\text{data}}$	data loss	Error between prediction and label, usually mean squared error.
$\mathcal{L}_{\text{phys}}$	physics loss	Additional penalty based on a physical property, such as divergence or a boundary condition.
λ_{phys}	physics weight	Nonnegative coefficient controlling the strength of the physics penalty relative to the data loss.
μ_j, s_j	scaler statistics	Mean and standard deviation of feature j, computed from training data only.
r	reduced rank	Number of POD modes retained in a reduced representation.
ϕ_k	POD mode	The kth spatial basis function. It is a field-shaped pattern learned from development snapshots.
a_k	modal coefficient	Scalar amplitude multiplying POD mode ϕ_k for a particular physical case.
σ_k	singular value	Strength of POD mode k in the snapshot dataset.

4.4 Additional notation for the Fokker–Planck closure track

Track 6 does not predict a final flow field directly. It learns the expensive local closure map used inside a particle Fokker–Planck solver. The symbols below are introduced here because they are different from the coordinate-DNN notation used in Tracks 1–5.

Symbol	Name	Meaning in Track 6
ρ	mass density	Cell mass density obtained from particle weights divided by cell volume.
T	translational temperature	Temperature obtained from the second central moment of molecular velocity.
U_i	bulk velocity	Cell-average molecular velocity in direction i.
Π_{ij}	second central moment / pressure tensor input	Six independent symmetric components used by the neural closure. In the supplied code the array is named `PIJ`.
q_i	heat-flux moment input	Three third-order central-moment components; the code stores them in `Q`.
$DM2$	scalar second central moment	$DM2 = \langle c_i c_i \rangle$, where $\mathbf{c} = \mathbf{V} - \mathbf{U}$ is peculiar velocity. It is proportional to temperature.
ν	relaxation/collision-frequency feature	Local scalar frequency available to both the exact and learned closure paths.
C_{ij}	stress-relaxation coefficients	Six symmetric coefficients of the cubic-FP drift. The code stores them in the first six output channels and in `coeffs['A']`.
Γ_i	heat-flux relaxation coefficients	Three coefficients controlling the heat-flux part of the drift. The code stores them in the last three output channels and in `coeffs['B']`.
$\mathbf{x}_{FP}$	closure-network input	The 16-component vector $(\rho, T, U_i, \Pi_{ij}, q_i, DM2, \nu)$.
$\mathbf{y}_{FP}$	exact closure target	The 9-component vector (C_{ij}, Γ_i) returned by the high-order moment assembly and exact local 9×9 solve.

A small coefficient-regression error is not the final objective. The trained coefficients are inserted back into the stochastic particle update, so Track 6 also requires a complete *closed-loop* cavity test. This distinguishes an offline regression benchmark from an operational physics–ML solver.

4.5 Training, validation, development, and blind test

The terms below have different purposes and must not be interchanged.

- **Training cases** update the network parameters θ.
- **Validation cases** are not used in gradient updates. They are used to select architecture, rank, loss weight, or stopping epoch.
- **Development set** means the training plus validation cases that are permitted during model design. After the design is frozen, the final model may be retrained using all permitted development cases.
- **Blind test cases** remain closed until all design choices are fixed. They estimate generalization to unseen physical cases.

A model is not truly blind-tested if the student changes architecture, rank, loss weight, or stopping rule after viewing the test result.

5 Core Concepts Used in Every Track

5.1 Full-order reference model and surrogate model

The Week-4 cavity solver is the **full-order reference model** for this project. It numerically solves the governing equations on a fixed grid and produces complete fields. A **surrogate model** is a cheaper approximation trained to reproduce the reference solution over a family of inputs.

The surrogate does not replace the physical equations in a universal sense. It approximates the mapping represented by the available dataset. If the reference solver contains grid error, pressure-recovery error, or limited accuracy at high Re, the surrogate learns those labels. Therefore, "small error against the dataset" and "physical truth" are not identical claims.

5.2 Coordinate DNN: turning fields into supervised samples

The coordinate network used in Week 4 learns

$$\widehat{\mathbf{y}} = f_\theta(Re, x, y), \qquad \widehat{\mathbf{y}} = (\hat{u}, \hat{v}, \hat{p}).$$

For every development Reynolds number, each grid point becomes one supervised row. The same network represents the entire field because spatial coordinates are part of the input.

Before training, each input feature is standardized using training statistics:

$$z_j^{\text{scaled}} = \frac{z_j - \mu_j}{s_j}.$$

Each output channel is also standardized so that u, v, and p contribute comparably to the loss. The scalers must be fitted on the training cases only; otherwise validation or test information leaks into training.

The baseline data loss is

$$\mathcal{L}_{\text{data}} = \frac{1}{N_b} \sum_{i=1}^{N_b} \|\widehat{\mathbf{y}}_i - \mathbf{y}_i\|_2^2 ,$$

where N_b is the number of point samples in a mini-batch. Minimizing this quantity improves average pointwise agreement, but it does not guarantee correct wall behavior, zero divergence, accurate vortex location, or reliable extrapolation.

5.3 Interpolation and extrapolation

Interpolation predicts inside the range covered by development cases. For example, training at $Re = 250$ and $Re = 300$ and testing at $Re = 275$ is interpolation. **Extrapolation** predicts outside the covered range, such as training only up to $Re = 300$ and testing at $Re = 400$.

Interpolation can still fail locally, especially near walls or weak secondary vortices. Extrapolation can look visually smooth while being quantitatively wrong. Therefore, both must be checked with reference fields, centerlines, and physical metrics.

5.4 Numerical error versus physical error

A global relative L_2 error measures average field disagreement. A physical check asks whether the prediction respects a property that matters for the flow. These two kinds of evidence answer different questions.

For example, a model may have a small global velocity error but a large wall error because most grid points are in the interior. Conversely, a strong divergence penalty may reduce divergence while slightly increasing velocity error. The project should report the tradeoff rather than hiding one metric.

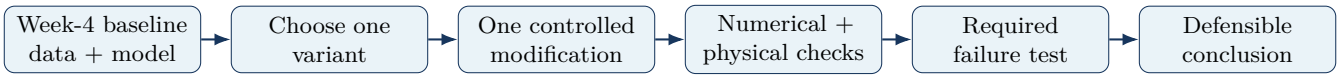
5.5 Controlled experiment

A controlled experiment changes one primary factor while keeping the comparison fair. The baseline and modified model should use the same development cases, blind cases, output normalization, architecture where possible, optimizer, seed policy, and metric definitions. The purpose is to attribute the observed change to the intended modification.

What counts as a scientific conclusion?

A conclusion is a bounded statement supported by the experiment, for example: "Within $100 \leq Re \leq 300$, direct field interpolation was more accurate than the tested coordinate DNN, while the DNN produced smoother but less accurate high-Re centerlines." A statement such as "neural networks are good for CFD" is too broad for the evidence.

6 The Guided Project Workflow



6.1 Six tracks, eleven project variants

Variant	Track	Scientific question	Level
1A	Reynolds generalization	Interpolation at unseen Reynolds numbers	Intermediate
1B	Reynolds generalization	Higher-Re extrapolation and failure boundary	Intermediate
2A	Physics-guided DNN	Does wall-weighted learning improve wall accuracy?	Intermediate
2B	Physics-guided DNN	Does divergence regularization improve incompressibility?	Advanced
3A	POD study	What is the smallest useful POD rank?	Intermediate
3B	POD study	Separate output bases or one shared scaled basis?	Intermediate/Advanced
4A	Uncertainty	How variable are repeated neural-network trainings?	Intermediate
4B	Data sufficiency	How many physical Reynolds-number cases are needed?	Intermediate
5A	Rarefied cavity	Do seed-averaged DSMC labels improve the surrogate?	Advanced
5B	Rarefied cavity	Can a model generalize to an unseen higher Kn?	Advanced
6A	Fokker–Planck neural closure	Can a q-weighted 16-to-9 closure surrogate retain held-out cavity fidelity when deployed inside the particle solver?	Advanced research / instructor approval

6.2 Project competition, prizes, and prize-track variants

Project competition and prize pathway

At the end of the term, the instructor will award prizes for the top three final projects: **First Prize: $500**, **Second Prize: $300**, and **Third Prize: $200**. A project may earn a high course grade without being competitive for a prize. Prize consideration is reserved for work with a realistic pathway to a **conference-quality submission** and, in the strongest cases, a **journal-quality continuation** if the student chooses to extend the study.

All eleven variants can receive full course credit. The variants below are the most natural *prize-track candidates* because they support a sharper research claim, a matched representation benchmark, or a direct continuation toward fluid/kinetic research.

Variant	Why it is a natural prize-track candidate
1B	A clear extrapolation and failure-boundary study can produce a defensible conference claim if the trustworthy and breakdown regimes are identified with matched baselines and physical evidence.
2B	A physics-guided loss can support a meaningful claim if divergence improves without unacceptable global-error degradation and the result is supported by a controlled ablation.
3A	POD-rank sensitivity directly studies compression, field fidelity, runtime/storage, and the loss of local physical structures. This is closely aligned with compact-representation research.
3B	Separate versus shared bases can become a matched-capacity representation study, especially if the student explains why a common basis succeeds or fails for velocity and pressure.
5A	Noise-aware learning from repeated kinetic simulations can separate surrogate error from sampling variability and is a strong basis for a rarefied-flow conference study.
5B	Generalization across Knudsen number can expose regime transfer and failure; with stronger reference data and extended validation it has a plausible journal continuation path.
6A	The project mirrors an active GPU-native Fokker–Planck closure study and combines data generation, q-weighted learning, native deployment, complete-condition testing, high-order diagnostics, and runtime analysis. It is the strongest direct conference/journal continuation track when executed at production resolution.

Projects inspired by *compact, physically faithful representations* are especially encouraged for the prize track. This means using matched-storage or matched-capacity baselines where appropriate, preserving quantities that matter physically rather than only smooth low-order fields, and making a claim about fidelity, compression, uncertainty, or regime transfer that remains meaningful beyond this course.

6.3 Minimum standard for prize consideration

A prize candidate must satisfy all normal course requirements and also reach the higher standard below.

1. **Clear research question.** The question is narrow, testable, and physically meaningful.
2. **Substantive extension beyond Week 4.** Changing only a plot, seed, or split is insufficient. The project adds a physics-guided objective, representation benchmark, uncertainty/noise study, or regime-transfer experiment.
3. **Matched baseline comparison.** The baseline and modification use the same permitted data, metric definitions, and as many identical training choices as possible.
4. **Multiple forms of evidence.** Report numerical metrics and at least **three** physical checks; at least

one must be local or structure-aware, such as wall behavior, vortex location, pressure centerlines, divergence, recirculation topology, or a moment-sensitive quantity.

5. **Ablation or sensitivity study.** Include at least one controlled parameter sweep, rank study, penalty study, case-count study, or matched-capacity comparison.

6. **Failure analysis.** Show a condition where the model becomes inaccurate or physically unreliable and explain the mechanism.

7. **Reproducibility.** The submitted notebook runs top-to-bottom with all required files and records the final configuration, seeds, versions, and data hash.

8. **Research-quality communication.** Figures are publication-style, and the report includes an abstract-style summary and a conclusion that states what was learned.

6.4 Conference-level and journal-path expectations

A project is **conference-level** when it makes one clean technical claim with adequate evidence, a matched baseline, and an honest scope/limitation statement. A project has a plausible **journal-path continuation** when the class result opens a broader question that can be extended with additional cases, stronger reference data, deeper validation, or a more capable method. The six-week project is not expected to become a paper immediately; the goal is for the strongest projects to become credible seeds for a conference abstract or later manuscript.

6.5 Optional stretch extensions for advanced students

The required project defines the common floor. Students who complete the required comparison early may request approval for **one** optional stretch extension listed under the selected track. Stretch work is not required for full course credit and should not be started until the Week-5 checkpoint is complete. Its purpose is to raise the ceiling for strong doctoral students and prize-track projects without increasing the entry barrier for the class. A stretch extension should add one new scientific dimension while preserving the original matched baseline and blind-test discipline.

7 What the Instructor Provides and What the Student Owns

7.1 Provided material

- a fixed Week-4 cavity dataset and quality-control protocol;
- a common file-check notebook, `P0_Project_Setup.ipynb`;
- one starter notebook for each project track;
- reusable loading, training, metric, POD, plotting, and mini-DSMC functions;
- for Track 6, a supplied particle-FP reference solver, exact-data generator, TensorFlow trainer, CuPy deployment wrapper, and closed-loop analyzer;
- a bounded set of model choices and experiment variables;
- required metrics, physical checks, and report templates.

7.2 Student responsibility

- choose one variant and state one research question;
- run and explain the supplied baseline;
- freeze the split and comparison plan before blind testing;
- make the required modification and no uncontrolled simultaneous changes;
- generate a reproducible table and three to five decision-relevant figures;
- perform at least two physical checks;
- identify and explain one failure or tradeoff;
- defend one important code cell and one scientific conclusion.

Central message

The project is not graded by the number of lines of new code. It is graded by the quality of the experiment, the validity of the comparison, the physical reasoning, and the student's ability to explain the implementation.

8 Minimum Viable Project

Every approved project must contain all eight items below.

1. A reproduced baseline from the supplied notebook.
2. One extension beyond Week 4.
3. A controlled baseline-versus-modification comparison.
4. A case-wise train/validation/test design.
5. At least two physical-validation checks.
6. One documented failure condition or tradeoff.
7. One metric table and three to five useful figures.
8. A short code-origin statement and a five-minute code defense.

Required deliverable

A project that produces a lower loss but does not include physical checks and a failure analysis is incomplete. A project whose modification does not outperform the baseline may still receive full credit if the experiment is correct and the negative result is explained honestly.

9 Project Timeline

9.1 Beginning of Week 5: setup submission

Run `P0_Project_Setup.ipynb` and submit:

- `project_choice.json`;
- the dataset audit;
- the Re=275 interpolation baseline table;
- the selected track notebook filename.

9.2 End of Week 5: mandatory checkpoint

Submit one ZIP containing:

1. an executable notebook with the baseline reproduced;
2. a frozen configuration/split cell;
3. one preliminary modified result;
4. one numerical comparison table;
5. one physical check;
6. a paragraph naming the planned failure test;
7. an AI/code-assistance log, even if the entry is "none."

9.3 End of Week 6: final project

Submit:

- one clean executed notebook;
- one PDF report of approximately 6–10 pages;
- all CSV/NPZ files needed to reproduce reported figures;
- `reproducibility.txt` with seed, package versions, runtime, and hardware;
- a 5–7 minute presentation or code/results discussion, as assigned.

10 File Organization and Colab Workflow

Use short filenames and one flat working folder in Colab.

```
project_folder/
  P0_Project_Setup.ipynb
  P2_Physics_Guided_DNN.ipynb # example selected notebook
  w4utils.py
  w5_common.py
  cavity_data.npz
  project_choice.json
  P2_results.csv
```

Track 5 additionally requires `mini_dsmc.py`. Track 6 uses the supplied `Track6_FP_Cavity` files and should be unpacked as one flat folder on a GPU runtime. Do not upload unrelated Colab sample datasets such as MNIST or California Housing.

To reduce Windows path-length problems and Colab upload friction, the instructor also provides one **flat ZIP** for setup and one for each track: `P0_Setup_v1_4_2.zip`, `Track1_Re_v1_4_2.zip`, `Track2_Physics_v1_4_2.zip`, `Track3_POD_v1_4_2.zip`, `Track4_Uncertainty_v1_4_2.zip`, `Track5_Kinetic_v1_4_2.zip`, and `Track6_FP_Closure_v1_4_2.zip`. Each track ZIP includes the required notebook and helper/data files; the Track-6 ZIP contains the separate FP reference solver and wrappers rather than the Week-4 continuum dataset.

11 Common Numerical and Physical Evidence

Every track uses the same philosophy: one scalar score is not enough. The final claim should be supported by global numerical error, local profile evidence, and physical checks related to the chosen research question.

11.1 Global relative velocity error

For a blind field on all grid points,

$$E_{uv} = \frac{\sqrt{\sum_{j=1}^{N_g}[(\hat{u}_j - u_j)^2 + (\hat{v}_j - v_j)^2]}}{\sqrt{\sum_{j=1}^{N_g}(u_j^2 + v_j^2)} + \varepsilon},$$

where j indexes grid points and ε is a very small number preventing division by zero. The numerator is the L_2 norm of the vector-velocity error; the denominator is the L_2 norm of the reference velocity.

This metric measures average field fidelity relative to the overall velocity magnitude. It can be dominated by the primary vortex and may underweight weak corner structures.

11.2 Pressure error and pressure gauge

Pressure is stored as zero-mean gauge pressure:

$$\frac{1}{N_g}\sum_{j=1}^{N_g} p_j = 0.$$

The global relative pressure error is

$$E_p = \frac{\|\hat{p} - p\|_2}{\|p\|_2 + \varepsilon}.$$

Because recovered pressure is especially sensitive near the discontinuous lid corners, the notebooks also report an **interior pressure error** after removing a thin boundary rim. This allows the student to distinguish interior field fidelity from corner sensitivity.

11.3 Mean absolute error and percentile error

For a scalar field q,

$$\mathrm{MAE}(q) = \frac{1}{N_g}\sum_{j=1}^{N_g} |\hat{q}_j - q_j|.$$

For vector velocity, the pointwise vector error is

$$e_j^{\mathrm{vec}} = \sqrt{(\hat{u}_j - u_j)^2 + (\hat{v}_j - v_j)^2}.$$

The 95th percentile of e_j^{vec} describes a high but not single-point-dominated error level. It is often more informative than the maximum error, which may be controlled by one corner node.

11.4 Centerline errors

Two standard cavity diagnostics are:

- vertical profile $u(x/L = 0.5, y/L)$;
- horizontal profile $v(x/L, y/L = 0.5)$.

A relative centerline error is computed using the same L_2 idea but only on profile points. Centerlines are valuable because two streamline plots may look similar even when the velocity distribution through the vortex is different.

11.5 Wall metrics

The stationary walls require no penetration and no slip in the continuum reference, while the top wall has $u = U_{\mathrm{lid}}$ away from the corners. The project wall metric combines wall velocity residuals. The four corner nodes are excluded because the moving-lid and stationary-wall conditions meet discontinuously there. With this convention, the CFD reference itself has zero wall error.

Students should report both a wall RMS metric and a lid-specific mean absolute error when wall behavior is relevant.

11.6 Divergence metric

For an incompressible prediction,

$$D(x, y) = \frac{\partial \hat{u}}{\partial x} + \frac{\partial \hat{v}}{\partial y}.$$

The notebooks report norms such as

$$D_{L_2} = \sqrt{\frac{1}{N_g} \sum_{j=1}^{N_g} D_j^2}, \qquad D_{L_\infty} = \max_j |D_j|.$$

The reference velocity obtained from the discrete streamfunction is divergence-free to machine precision under the corresponding central-difference operators. A DNN trained only on pointwise data generally is not.

11.7 Vortex and recirculation evidence

A cavity field should be checked for the direction and location of the primary recirculation. Possible diagnostics include:

- sign of the lower-cavity return flow;
- approximate primary-vortex center;
- presence or loss of weak corner/secondary recirculation;
- streamline topology and consistency with centerlines.

These checks are often qualitative plus quantitative. A project should avoid claiming a precise vortex location when the reference grid or stochastic field does not support that precision.

11.8 How to choose physical checks

Choose checks that directly test the project claim.

- Track 1: centerlines, pressure, wall behavior, and failure versus Re.
- Track 2A: wall RMS and lid error, plus interior/global error.
- Track 2B: divergence, plus field and centerline fidelity.
- Track 3: low-rank structure preservation, centerlines, pressure, and wall/weak-vortex behavior.
- Track 4: stability of physical features across seeds or case sets.
- Track 5: temperature positivity, lid-driven sign, return-flow sign, seed variability, and Kn trend.

11.9 Reference-data accuracy and scope

The supplied continuum fields are the fixed Week-4 teaching reference, not an exact Navier–Stokes benchmark. Against the Ghia et al. centerline data, the stored $Re = 100$ case has approximately $E_u = 0.9\%$ and $E_v = 2.4\%$; the $Re = 400$ case has approximately $E_u = 5.2\%$ and $E_v = 15.4\%$. These cases pass the Week-4 quality gate, but conclusions must be described as agreement with the *Week-4 reference solver*. A smoother surrogate cannot be assumed to be more physically accurate than its labels.

11.10 A controlled experiment

A controlled comparison changes one primary factor while holding the following fixed whenever possible: physical cases, blind split, model architecture, optimizer, seed policy, point stride, stopping rule, normalization, and metric definitions. If several factors change together, the project must acknowledge the confounding and avoid assigning the result to one cause.

12 Responsible Use of AI Coding Assistance

AI assistance is permitted only under the course policy. It does not replace understanding.

Required deliverable

Include a table with three columns: **AI-assisted item**, **what was changed**, and **how it was verified**. If no AI assistance was used, state that explicitly.

Students must be able to:

- explain one model-building cell line by line;
- change one parameter during a code defense;
- predict the qualitative effect before rerunning;
- identify the shapes and physical meanings of the main arrays;
- explain why the test split is scientifically valid.

13 Choosing a Project Without Starting from a Blank Page

Use the questions below in order.

1. Do you want to study **where a model generalizes or fails as the physical parameter changes**? Choose Track 1.
2. Do you want to learn how **physics enters the training objective**? Choose Track 2.
3. Do you want to understand **field compression and reduced-order models**? Choose Track 3.
4. Do you want to study **reproducibility, uncertainty, or data sufficiency**? Choose Track 4.
5. Do you want a more advanced project involving **stochastic kinetic data and Knudsen number**? Choose Track 5 with instructor approval.
6. Are you assigned to the instructor's **GPU-native Fokker–Planck neural-closure research** and prepared to use a Colab/Unity GPU? Choose Track 6 with explicit instructor approval.

Important boundary

Choose the scientific question first, then choose the model. Do not choose a project only because the model name sounds advanced. A simple model used in a well-controlled experiment is stronger than a complicated model with no defensible split or physical test.

13.1 Examples of acceptable one-sentence questions

- At what Reynolds number does a low-Re coordinate DNN cease to reproduce the blind centerline profile within 5%?
- Can a wall-weighted loss reduce lid and side-wall error without increasing interior velocity error by more than 10%?
- What is the smallest POD rank that preserves both pressure and velocity centerlines at Re=275?
- Does a five-seed ensemble provide an uncertainty map that correlates with actual blind-field error?
- Does averaging two independent stochastic cavity runs produce a more transferable rarefied-flow surrogate than using one seed?
- Does q-weighting reduce held-out heat-flux-coefficient error without unacceptable stress-block degradation, and does that tradeoff survive a complete 800 m/s closed-loop Fokker–Planck cavity test?

13.2 Examples that are too broad

- "Use AI to solve fluid mechanics."
- "Compare all neural networks."
- "Make DeepONet better."
- "Build a digital twin of a cavity."
- "Use ChatGPT to generate a PINN."

These must be narrowed to one input range, one baseline, one controlled modification, and one measurable claim.

14 Notebook and File Map

File	Used by	Purpose
`P0_Project_Setup.ipynb`	Everyone	Checks the Week-4 files and reproduces a common non-neural baseline.
`P1_Re_Generalization.ipynb`	1A/1B	Split design, architecture selection, interpolation and extrapolation evidence.
`P2_Physics_Guided_DNN.ipynb`	2A/2B	Wall sample weighting or automatic-differentiation divergence penalty.
`P3_POD_Study.ipynb`	3A/3B	POD rank study and separate/shared field bases.
`P4_Uncertainty_Study.ipynb`	4A/4B	Repeated training seeds or physical-case sufficiency.
`P5_Rarefied_Cavity.ipynb`	5A/5B	Stochastic kinetic labels and unseen-Kn testing.
`P6_FP_Cavity_Closure.ipynb`	6A	Exact cubic-FP data generation, uniform/q-weighted closure training, model export, and complete closed-loop cavity testing.
`Track6_FP_Cavity/*.py`	6A	Supplied reference solver, bounded configuration, data generator, TensorFlow trainer, CuPy test wrapper, and field analyzer.
`w4utils.py`	Tracks 1–4	Week-4 data loading and standard field/physics metrics.
`w5_common.py`	Tracks 1–5	Common model, POD, plotting, and experiment helper functions.
`mini_dsmc.py`	Track 5	CPU-friendly teaching molecular cavity simulator.
`qa_self_test.py`	Optional QA	Checks versions, data hash, truth wall metrics, pooled temperature, and the Keras smoke path when TensorFlow is available.
`cavity_data.npz`	Tracks 1–4	Fixed Week-4 continuum cavity fields.

Central message

The notebooks are intentionally not empty templates. They contain a working baseline and the complete infrastructure needed for the project. Students edit only the configuration and experiment cells identified in the notebook, then add their own interpretation and optional diagnostic plots.

Important boundary

The workload estimates listed below refer to coding, simulation, and analysis after the supplied files run successfully. Most students should budget an additional 3–5 hours for the final 6–10 page report, figure cleanup, reproducibility check, and code-defense preparation.

15 Track 1: Reynolds-Number Generalization

15.1 What this track teaches

Notebook: `P1_Re_Generalization.ipynb`. This track studies how a surrogate behaves when the Reynolds number changes. The Week-4 coordinate DNN already learned the pointwise mapping

$$(Re, x, y) \longmapsto (u, v, p).$$

The project does not ask the student to invent a new network. It asks the student to design a scientifically valid generalization experiment and decide when the prediction should or should not be trusted.

Reynolds number in this project

$$Re = \frac{U_{\text{lid}} L}{\nu}.$$

Here U_{lid} is the moving-lid speed, L is the cavity length, and ν is the kinematic viscosity. As Re increases, viscous diffusion becomes less dominant, gradients become sharper, and weak corner structures can change. The neural network is therefore not simply interpolating a number; it is interpolating a family of spatial flow fields.

Required deliverable

Starting files: `P1_Re_Generalization.ipynb`, `w4utils.py`, `w5_common.py`, and `cavity_data.npz`.
Typical experimental workload: 6–9 hours after the files run; report preparation is additional.
Student edits: one variant, one frozen split, a bounded architecture list, one predeclared failure criterion, and one case for detailed analysis.

15.2 The two models being compared

15.2.1 Direct field interpolation

Suppose the target Reynolds number Re_* lies between two available development cases $Re_a < Re_* < Re_b$. For any field $q \in \{u, v, p\}$, direct linear interpolation is

$$\widehat{q}_{\text{int}}(Re_*, x, y) = (1 - \alpha) q(Re_a, x, y) + \alpha q(Re_b, x, y), \qquad \alpha = \frac{Re_* - Re_a}{Re_b - Re_a}.$$

This baseline uses complete fields at neighboring Reynolds numbers. Because the supplied cavity family is smooth and uses the same grid for every case, interpolation can be extremely accurate. It is intentionally included as a difficult baseline: the purpose is not to guarantee that the DNN wins.

15.2.2 Coordinate DNN

The DNN receives one point at a time:

$$\widehat{\mathbf{y}} = f_\theta(Re, x, y), \qquad \widehat{\mathbf{y}} = (\hat{u}, \hat{v}, \hat{p}).$$

Unlike direct field interpolation, the DNN stores a set of network parameters rather than complete neighboring fields. It may be useful if one wants continuous evaluation at arbitrary coordinates or a model that can later be extended to more input parameters. However, those potential advantages do not excuse a larger error.

15.3 Meaning of the split variables in the notebook

Code variable	Meaning	Rule
`TRAIN_RE`	Reynolds cases used in gradient updates during architecture selection	May not contain blind cases.
`VAL_RE`	One case used to select architecture and stopping epoch	May influence design, but never gradient updates in the selection stage.
`DEV_RE`	All permitted development cases, equal to training plus validation	Used for the final retraining after architecture and epoch budget are frozen.
`TEST_RE`	Blind Reynolds cases	Must remain unopened until all design choices are fixed.
`best_epoch`	Epoch selected by validation behavior	Reused as a fixed budget when retraining on all development cases.

The final DNN and the interpolation baseline must have access to the same permitted development cases. Otherwise, the comparison is confounded.

15.4 Variant 1A: interpolation at unseen interior Reynolds numbers

Research question for Variant 1A

How accurately can a model predict complete cavity fields at Reynolds numbers located *inside* the development range but absent from training, and which local flow features fail before the global field error becomes large?

A typical design spans the range with development cases and reserves one or more interior cases for blind testing. The student should compare direct field interpolation and the coordinate DNN at the same blind Re values.

What the student does:

1. Freeze `TRAIN_RE`, `VAL_RE`, and `TEST_RE` before training.
2. Select the network architecture only from validation evidence.
3. Retrain the selected architecture on all development cases for the frozen epoch budget.
4. Evaluate interpolation and DNN with the same global and physical metrics.
5. Identify one local feature that is more difficult than the global field, such as the lid region, pressure profile, or weak corner recirculation.

A strong result may be negative. For example, the correct conclusion may be that direct interpolation is both simpler and more accurate for this smooth, fixed-grid dataset.

15.5 Variant 1B: extrapolation and the failure boundary

Research question for Variant 1B

When the DNN is trained only on lower Reynolds numbers, at what higher Reynolds number does it first violate a predeclared numerical or physical reliability criterion?

The student trains on a lower-Re development family and tests at larger Re values. The purpose is not to present a smooth extrapolated field as proof of success. The purpose is to locate a **failure boundary**.

Before blind testing, define a criterion such as

$$E_{uv} > 0.05,$$

or a centerline error above 5%, or a wall-error threshold justified from the baseline. The first blind case that exceeds the criterion defines the observed failure boundary for the tested model and dataset.

Important boundary

A network can produce visually smooth streamlines far outside its training range. Smoothness is a property of the network, not evidence of physical validity. Extrapolation claims require a reference case.

15.6 Step-by-step notebook workflow

1. **Audit the dataset.** Confirm the available Reynolds numbers, grid size, and fields.
2. **Declare the split.** Write the scientific claim and failure criterion before training.
3. **Assemble pointwise samples.** Flatten complete development fields only after the case labels are assigned.
4. **Fit scalers.** Compute input and output means/standard deviations from training data only.
5. **Select architecture.** Compare the bounded candidate list on the validation case.
6. **Freeze the design.** Store the selected architecture and `best_epoch`.
7. **Retrain on all development cases.** Do not keep the former validation case out of the final permitted fit.
8. **Open blind cases.** Evaluate interpolation and DNN once with the frozen design.
9. **Analyze local evidence.** Plot centerlines, pressure, walls, divergence, and an error map.

Example configuration

```
VARIANT = "1B"
TRAIN_RE = [100, 150, 200, 225, 250]
VAL_RE = 300
TEST_RE = [350, 375, 400]
FAILURE_THRESHOLD = 0.05 # declared before blind testing
```

15.7 Required numerical and physical evidence

- validation architecture table and selected epoch budget;
- global relative velocity and pressure errors for every blind case;
- vertical u centerline and horizontal v centerline;
- wall metric and divergence metric;
- one pressure contour/profile comparison;
- error versus Reynolds number;
- direct interpolation versus DNN under identical development information;
- a bounded statement of the reliable Reynolds-number range.

15.8 How to interpret common outcomes

- **Interpolation wins strongly:** explain that the field family is smooth, fixed-grid, and densely bracketed in Re. Discuss whether the DNN offers any other practical advantage.
- **DNN matches interpolation globally but loses wall accuracy:** global MSE is not resolving the most sensitive region.
- **DNN extrapolates smoothly but centerlines drift:** the model is outside its supported physical range even if contours appear plausible.
- **Both methods fail at the same high Re:** the development dataset may not contain enough information about the emerging high-Re structure.

Required failure test

For 1A, identify an interior case where a local wall, pressure, or weak-vortex feature is lost. For 1B, report the first Reynolds number that violates the predeclared reliability criterion and explain the physical/numerical symptom.

15.9 Optional Stretch Extensions (advanced / prize track)

Complete the required study first. With instructor approval, choose at most one:

1. train a small ensemble and test whether ensemble spread increases near the observed failure boundary;
2. create a validation-derived rejection rule and test whether it flags unreliable blind cases;
3. compare interpolation, coordinate DNN, and a reduced-order coefficient model using identical development cases and matched runtime/storage reporting.

15.10 Track-1 Required Output Package

- `P1_results.csv`;
- validation architecture table;

- error-versus-Re plot;
- one detailed blind-case figure;
- interpolation-versus-DNN table for every blind case;
- a paragraph defining the reliable range;
- a paragraph explaining one failure or local defect.

15.11 Frequent Track-1 Mistakes

- calling a case inside the development range "extrapolation";
- changing architecture after viewing blind results;
- giving interpolation access to more Reynolds cases than the DNN;
- claiming success from streamline similarity while centerlines or pressure are inaccurate;
- querying an unsupported Re without a reference field and treating smooth output as truth;
- reporting only the model that wins and omitting the strong baseline.

16 Track 2: Physics-Guided Coordinate DNN

16.1 What this track teaches

Notebook: `P2_Physics_Guided_DNN.ipynb`. Week 4 trained a coordinate DNN using one uniform data loss. Track 2 asks a deeper question: *what should the optimizer care about?* A model may fit most grid points well while violating a boundary condition or incompressibility. Physics-guided learning adds a carefully defined preference to the training objective and then measures the resulting tradeoff.

The baseline model and data split remain fixed. The primary experimental factor is the loss formulation.

Required deliverable

Starting files: `P2_Physics_Guided_DNN.ipynb`, `w4utils.py`, `w5_common.py`, and `cavity_data.npz`.
Typical experimental workload: 7–10 hours for 2A and 9–12 hours for 2B; report preparation is additional.
Student edits: one small predeclared weight list, the primary validation metric, and the global-error tolerance used during model selection.

16.2 Baseline data loss

Let N_b be the number of point samples in one training batch. For output vector $\mathbf{y}_i = (u_i, v_i, p_i)$ and prediction $\widehat{\mathbf{y}}_i$, the basic loss is

$$\mathcal{L}_{\text{data}} = \frac{1}{N_b} \sum_{i=1}^{N_b} \|\widehat{\mathbf{y}}_i - \mathbf{y}_i\|_2^2 .$$

In the notebook, the output channels are standardized before this loss is evaluated. This avoids allowing the numerically largest channel to dominate. After training, all reported physical metrics are computed after reversing the standardization.

Uniform MSE treats every point equally. That is often reasonable, but it does not know that the moving lid, no-penetration walls, or divergence may be especially important for a fluid-mechanics application.

16.3 Variant 2A: wall-weighted learning

Research question for Variant 2A

Can a spatially weighted data loss improve near-wall velocity accuracy without increasing the global blind-field error beyond an acceptable tolerance?

For a point (x, y) in the unit square, define the distance to the nearest wall:

$$d_w(x, y) = \min(x, 1 - x, y, 1 - y).$$

The notebook assigns the sample weight

$$w(x, y) = 1 + A \exp\left[- \left(\frac{d_w}{\delta} \right)^2 \right].$$

The variables are:

- $A \geq 0$: wall-weight amplitude. $A = 0$ recovers the baseline. Larger A gives near-wall points more influence.

- $\delta > 0$: thickness of the weighted region. Small δ concentrates the extra weight very close to the wall; larger δ affects a broader band.
- d_w: nondimensional wall distance.

The weighted loss is

$$\mathcal{L}_{\text{wall}} = \frac{\sum_i w_i \, \|\hat{\mathbf{y}}_i - \mathbf{y}_i\|_2^2}{\sum_i w_i}.$$

The target values and architecture are unchanged. Only the relative importance of spatial samples changes.

Wall-weight code

```
def wall_weights(X, amplitude, thickness=0.10):
    d = wall_distance(X) # d_w for each point
    return 1.0 + amplitude*np.exp(-(d/thickness)**2)
```

What the student should learn:

1. A weighted loss changes the training objective, not the physics of the labels.
2. Lower wall error may come with larger interior error.
3. The wall metric excludes the four corner nodes because the moving-lid and stationary-wall conditions are discontinuous there.
4. A useful weight is not necessarily the largest tested weight.

16.4 Variant 2B: divergence-penalized learning

Research question for Variant 2B

Can an incompressibility penalty reduce the divergence of a coordinate DNN while preserving the accuracy of the velocity and pressure fields?

For an incompressible two-dimensional velocity field,

$$\nabla \cdot \mathbf{u} = \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0.$$

The physics-guided objective is

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda_{\text{div}} \mathcal{L}_{\text{div}},$$

where

$$\mathcal{L}_{\text{div}} = \frac{1}{N_b} \sum_{i=1}^{N_b} \left(\frac{\partial \hat{u}}{\partial x} + \frac{\partial \hat{v}}{\partial y} \right)_i^2.$$

Here $\lambda_{\text{div}} \geq 0$ controls the strength of the divergence penalty. $\lambda_{\text{div}} = 0$ is the baseline.

16.4.1 How automatic differentiation is used

The network output is differentiable with respect to its coordinate inputs. Keras/TensorFlow records the operations used to compute $\hat{u}$ and $\hat{v}$, then applies the chain rule to obtain coordinate derivatives. This is automatic differentiation (AD); it is not a finite-difference approximation of the network output.

However, the network sees standardized coordinates and predicts standardized outputs. If

$$\widetilde{x} = \frac{x - \mu_x}{s_x}, \qquad \widetilde{u} = \frac{u - \mu_u}{s_u},$$

then

$$\frac{\partial u}{\partial x} = \frac{s_u}{s_x}\frac{\partial \widetilde{u}}{\partial \widetilde{x}}.$$

Similarly,

$$\frac{\partial v}{\partial y} = \frac{s_v}{s_y}\frac{\partial \widetilde{v}}{\partial \widetilde{y}}.$$

The supplied custom trainer includes these scale factors. Omitting them would penalize a quantity whose numerical value depends on arbitrary standardization.

Penalty versus exact constraint

A divergence penalty is a *soft constraint*: the optimizer is encouraged, but not forced, to reduce divergence. An exactly divergence-free parameterization, such as predicting a streamfunction and differentiating it to obtain velocity, is a *hard structural constraint*. The required Track 2B project uses the soft penalty; the hard version is an optional extension.

16.5 Why a physics term can make the prediction worse

The optimizer must balance two objectives. If λ_{div} is too small, the result is almost identical to the baseline. If it is too large, the network may reduce divergence by producing an overly smooth or nearly constant velocity field. Therefore, the project should show a **tradeoff curve**, not only one selected setting.

16.6 Validation-constrained model selection

Let E_0 be the baseline validation relative velocity error and let ϵ be a tolerance declared before blind testing. The starter uses $\epsilon = 0.10$, meaning that a physics-guided model is rejected if its global validation error increases by more than 10% relative to the baseline.

Among settings satisfying

$$E_{uv}^{\text{val}} \leq (1 + \epsilon)E_0,$$

select the model with the lowest intended physics metric. For 2A, that metric is the wall error. For 2B, it is divergence. If no nonzero weight passes the global-error constraint, the correct conclusion is that the tested physics modification did not provide an acceptable tradeoff.

16.7 Step-by-step notebook workflow

1. Use the fixed Week-4 development and blind cases.
2. Train the zero-weight baseline.
3. Declare a small list of wall amplitudes or divergence weights.
4. Train every setting with the same architecture, optimizer, seed policy, and stopping rule.
5. Compute validation global error and the intended physical metric.
6. Apply the predeclared tolerance rule.
7. Freeze the selected weight before opening blind cases.

8. Compare baseline and selected model on global, local, and physical metrics.
9. Include one too-weak and one too-strong setting in the report.

16.8 Variables students are allowed to change

Variable	Track	Meaning
`WALL_AMPLITUDES`	2A	Small list of A values controlling near-wall emphasis.
`WALL_THICKNESS`	2A	δ, the width of the weighted wall region; change only with justification.
`DIV_WEIGHTS`	2B	Small list of λ_{div} values.
`GLOBAL_ERROR_TOL`	2A/2B	Allowed relative increase in validation global velocity error.
`PRIMARY_METRIC`	2A/2B	Wall error for 2A or divergence for 2B.

16.9 Required evidence

- table of every tested physics weight and validation metrics;
- baseline and CFD-truth reference rows;
- tradeoff plot: physical metric versus global error;
- blind relative velocity/pressure errors;
- centerline comparison;
- wall or divergence map/metric;
- explanation of the loss term and all symbols;
- one too-weak and one too-strong setting;
- conclusion stating whether the physics term helped the intended use case.

16.10 How to interpret common outcomes

- **Wall error improves and global error is unchanged:** the weighted objective successfully redirected capacity toward the wall region.
- **Divergence improves but centerlines worsen:** the penalty improved one physical property at the cost of field fidelity.
- **The largest weight wins only because the selection rule ignores global error:** the experimental design is invalid; use the tolerance guard.
- **No nonzero weight passes the guard:** report that the tested formulation did not improve the tradeoff. This is a valid result.

Required failure test

Find one setting that is too weak and one that is too strong. Explain the failure mechanism using both the loss definition and the flow field, not only the final scalar metric.

16.11 Optional Stretch Extensions (advanced / prize track)

Complete the required Track-2 study first. With instructor approval, choose at most one:

1. predict a streamfunction and obtain velocity by AD, giving an exactly divergence-free parameterization, then compare fairly with the penalty model;
2. combine wall weighting and divergence regularization in a two-factor ablation while preserving the global-error tolerance;
3. compare constant and scheduled physics weights and determine whether optimization path affects the final tradeoff.

16.12 Track-2 Required Output Package

- `P2_results.csv`;
- validation table for all weights;
- global-error/physics-metric tradeoff plot;
- blind field and centerline evidence;
- a short derivation or explanation of the added loss;
- one too-weak and one too-strong setting;
- a bounded conclusion about usefulness.

16.13 Frequent Track-2 Mistakes

- changing architecture while changing the physics weight;
- selecting a weight after viewing blind results;
- computing derivatives from standardized outputs without restoring physical scaling;
- assuming lower divergence guarantees lower velocity error;
- reporting only the selected model and hiding the tradeoff;
- using a large penalty that produces a nearly constant field;
- treating a soft penalty as an exact physical guarantee.

17 Track 3: POD and Reduced-Order Learning

17.1 Why reduced-order modeling is useful

Notebook: `P3_POD_Study.ipynb`. A full cavity solution contains thousands of values. For a 65×65 grid,

$$N_g = 65 \times 65 = 4225,$$

and the three fields (u, v, p) contain $3N_g = 12675$ values for one Reynolds-number case. A reduced-order model (ROM) seeks a lower-dimensional description that preserves the dominant field structure.

The key assumption is that the family of cavity fields does not fill the entire 12675-dimensional output space. As Re varies over a limited range, many fields may lie near a much smaller subspace. Proper Orthogonal Decomposition (POD) finds an efficient linear basis for that subspace.

Full-order model and reduced-order model

A **full-order model (FOM)** resolves all grid degrees of freedom and produces the complete field. A **reduced-order model (ROM)** represents the field using a small set of coordinates, called modal coefficients. The ROM is useful only if the reduced coordinates retain the physical features needed for the intended task.

Required deliverable

Starting files: `P3_POD_Study.ipynb`, `w4utils.py`, `w5_common.py`, and `cavity_data.npz`.
Typical experimental workload: 6–9 hours; report preparation is additional.
Student edits: one approved variant, a bounded rank list, one rank-selection rule, and one detailed low-rank failure comparison.

17.2 Step 1: turn fields into snapshot vectors

Consider one scalar field q, such as u. For development case i, flatten the $N_y \times N_x$ array into a vector

$$\mathbf{q}^{(i)} \in \mathbb{R}^{N_g}.$$

If there are M development cases, the mean field is

$$\overline{\mathbf{q}} = \frac{1}{M} \sum_{i=1}^{M} \mathbf{q}^{(i)}.$$

The centered snapshot matrix is

$$\mathbf{Q} = \begin{bmatrix} (\mathbf{q}^{(1)} - \overline{\mathbf{q}})^T \\ (\mathbf{q}^{(2)} - \overline{\mathbf{q}})^T \\ \vdots \\ (\mathbf{q}^{(M)} - \overline{\mathbf{q}})^T \end{bmatrix} \in \mathbb{R}^{M \times N_g}.$$

Each row is one complete centered field. Blind test fields must not appear in $\mathbf{Q}$, because the POD basis is part of the learned representation.

17.3 Step 2: singular value decomposition and POD modes

Compute the singular value decomposition (SVD)

$$\mathbf{Q} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T.$$

The matrices and symbols mean:

- $\mathbf{U}$: describes how the development snapshots combine the modes;
- $\mathbf{\Sigma} = \mathrm{diag}(\sigma_1, \sigma_2, \ldots)$: singular values ordered from largest to smallest;
- $\mathbf{V}$: contains orthonormal spatial directions;
- $\boldsymbol{\phi}_k = \mathbf{V}_{:,k}$: the kth POD mode, reshaped back to an $N_y \times N_x$ field for plotting.

The first mode captures the largest remaining variation around the mean field. The second mode captures the largest variation orthogonal to the first, and so on. POD is optimal among linear rank-r representations in the snapshot L_2 sense, but that optimality concerns average squared reconstruction error, not every physical feature.

17.4 Step 3: modal coefficients and rank-r reconstruction

The coefficient of mode k for case i is

$$a_k^{(i)} = (\mathbf{q}^{(i)} - \overline{\mathbf{q}})^T \boldsymbol{\phi}_k.$$

Keeping only the first r modes gives the rank-r reconstruction

$$\widehat{\mathbf{q}}_r^{(i)} = \overline{\mathbf{q}} + \sum_{k=1}^{r} a_k^{(i)} \boldsymbol{\phi}_k.$$

The integer r is the **POD rank**. Increasing r increases representational capacity, storage, and the number of coefficients that must be predicted.

Because the snapshots are mean-centered, the maximum nonzero rank is at most $M-1$. In the architecture-selection stage of the supplied notebook there are seven training snapshots, so at most six centered directions are available. This is why the required rank list stops at 6.

17.5 Step 4: cumulative POD energy

The cumulative energy captured by the first r modes is

$$\eta_r = \frac{\sum_{k=1}^{r} \sigma_k^2}{\sum_{k=1}^{r_{\max}} \sigma_k^2}.$$

A value such as $\eta_r = 0.999$ means that the retained modes explain 99.9% of the squared variation in the development snapshots. It does *not* guarantee that the model preserves a weak corner vortex, a wall gradient, or a pressure feature. A small-amplitude feature may contribute little to total energy but still be physically important.

Important boundary

Do not select the rank from cumulative energy alone. The selected rank must also satisfy blind or validation field metrics and at least one structure-sensitive physical check.

17.6 Step 5: the branch model predicts coefficients

POD compresses a known field, but a surrogate must predict the field at an unseen Reynolds number. The notebook trains a small branch network

$$\mathbf{a}(Re) = g_\theta(Re),$$

where $\mathbf{a}$ is the vector of retained modal coefficients.

For separate rank r bases for u, v, and p, the branch output is

$$\mathbf{a}_{\text{sep}}(Re) = [a_1^{(u)}, \ldots, a_r^{(u)}, a_1^{(v)}, \ldots, a_r^{(v)}, a_1^{(p)}, \ldots, a_r^{(p)}]^T \in \mathbb{R}^{3r}.$$

The field is reconstructed by inserting these predicted coefficients into the POD expansion.

This model is sometimes described as POD–DeepONet-style because fixed spatial modes play a trunk-like role and a neural branch predicts coefficients. In this course, the branch input is only the scalar Re, so it is more precise to call it a **parameterized POD neural surrogate**, not a general function-to-function operator learner.

17.7 Two distinct error sources

A POD surrogate can fail for two different reasons.

17.7.1 Basis/reconstruction error

Project the true reference field onto the retained modes using the true coefficients. The reconstruction error is

$$E_{\text{rec}}(r) = \frac{\|\mathbf{q} - \widehat{\mathbf{q}}_{r,\text{true coeff}}\|_2}{\|\mathbf{q}\|_2}.$$

This error asks whether rank r is large enough, assuming perfect coefficient knowledge.

17.7.2 Coefficient-prediction error

Use the branch-predicted coefficients:

$$E_{\text{ROM}}(r) = \frac{\|\mathbf{q} - \widehat{\mathbf{q}}_{r,\text{pred coeff}}\|_2}{\|\mathbf{q}\|_2}.$$

If E_{rec} is small but E_{ROM} is large, the basis is adequate but the coefficient model is poor. If both are large, the rank or linear-subspace assumption is inadequate.

Minimal reconstruction-error calculation

```
# F is a true flattened field, mean is the POD mean,
# and modes[:rank] contains the retained spatial modes.
true_coeff = (F - mean) @ modes[:rank].T
F_reconstructed = mean + true_coeff @ modes[:rank]
relative_reconstruction_error = np.linalg.norm(F_reconstructed-F) / np.linalg.norm(F)
```

17.8 Separate bases versus one shared scaled basis

17.8.1 Separate representation

Build independent POD decompositions for u, v, and p. Each variable receives modes adapted to its own spatial structure and amplitude. At rank r, the branch predicts $3r$ coefficients.

17.8.2 Shared scaled representation

First scale each field block by a development-set scale s_q, then concatenate:

$$\mathbf{z}^{(i)} = \left[\frac{\mathbf{u}^{(i)}}{s_u}, \frac{\mathbf{v}^{(i)}}{s_v}, \frac{\mathbf{p}^{(i)}}{s_p}\right].$$

A single POD basis is computed for the concatenated vector. Scaling is essential; without it, the field with the largest numerical variance would dominate the SVD.

A shared rank-r basis predicts only r coefficients, whereas three separate rank-r bases predict $3r$ coefficients. Therefore, equal nominal rank is *not* equal coefficient capacity. The required Variant 3B comparison should report both rank and coefficient count. A stronger matched-capacity study is an optional extension and may be limited by the small number of available snapshots.

17.9 What compression means

There are two different compression statements.

17.9.1 Online output-dimension reduction

A full three-field output has $3N_g$ values. Separate rank r POD predicts $3r$ coefficients, so the online output-dimension reduction is

$$C_{\text{output}} = \frac{3N_g}{3r} = \frac{N_g}{r}.$$

For $N_g = 4225$ and $r = 4$, the branch predicts 12 coefficients instead of 12675 field values, an output-dimension reduction of about 1056.

17.9.2 Archive/storage compression

The POD basis and mean fields must also be stored. For M raw three-field cases,

$$S_{\text{raw}} = 3MN_g$$

floating-point values. A separate rank-r POD archive using stored coefficients requires approximately

$$S_{\text{POD}} = 3N_g + 3rN_g + 3Mr.$$

With only eight cases and rank 4, this archive compression is modest because the modes themselves are large. POD becomes more attractive for many cases or when the basis supports continuous coefficient prediction. Students must distinguish output-dimension reduction from total-storage compression.

17.10 Variant 3A: POD-rank sensitivity

Research question for Variant 3A

What is the smallest POD rank that preserves the global fields and the specific physical structures required for blind cavity prediction?

Compare ranks 1–6. For every rank, report cumulative energy, reconstruction error, branch-prediction error, wall/divergence metrics, and at least one centerline or structure-sensitive check. Select the smallest scientifically adequate rank, not automatically the largest rank.

Required controlled factors: same development split, same branch architecture, same branch optimizer, and same metrics at every rank.

17.11 Variant 3B: separate versus shared basis

Research question for Variant 3B

Does a single scaled shared basis capture the coupled variation of velocity and pressure efficiently, or do the fields require separate spatial bases?

Compare the supplied separate and shared representations at each rank. Report the different coefficient counts. Analyze which field benefits or suffers from sharing. Pressure and velocity may not prefer the same modes because their spatial patterns and amplitudes differ.

A careful conclusion might be: "The shared basis achieved acceptable velocity compression with fewer coefficients, but pressure error increased because pressure-specific modes were not retained early enough."

17.12 Step-by-step notebook workflow

1. Freeze development, validation, and blind cases.
2. Construct POD modes from training snapshots only during selection.
3. Plot singular values and cumulative energy.
4. Compute true projected coefficients for development/validation cases.
5. Train the same branch MLP for every rank and basis family.
6. Compute reconstruction error and full ROM prediction error separately.
7. Freeze rank/basis with a stated compression-accuracy-physics rule.
8. Rebuild the basis using all permitted development cases.
9. Open blind cases and evaluate global and physical evidence.
10. Plot a deliberately low-rank failure beside the selected model.

17.13 A defensible rank-selection rule

One example is:

1. find the best validation velocity error $E_{\min}$ over all tested ranks;
2. keep ranks satisfying $E_{uv} \leq 1.10 E_{\min}$;
3. require pressure and physical metrics to remain below predeclared tolerances;
4. choose the smallest eligible rank.

This is a multi-objective rule: accuracy, physical fidelity, and compactness all matter.

Required failure test

Use a deliberately low rank and identify the first important feature that disappears: centerline curvature, pressure gradient, wall response, primary-vortex location, or weak corner structure. Explain why cumulative energy did not fully protect that feature.

17.14 Required evidence

- singular-value and cumulative-energy curves;
- table of rank, basis, coefficient count, validation error, wall error, divergence, and pressure error;
- reconstruction error separated from branch-prediction error;
- output-dimension reduction and, where meaningful, archive-storage estimate;
- low-rank versus selected-rank field/centerline comparison;
- blind-case evidence at all test Reynolds numbers;
- explanation of why the selected representation is scientifically adequate.

17.15 How to interpret common outcomes

- **Energy is near 100% but a corner feature is lost:** the weak feature contributes little to the global variance.
- **Reconstruction is excellent but prediction is poor:** the branch does not learn coefficient variation in Re accurately.
- **Shared basis uses fewer coefficients but pressure worsens:** the coupled basis allocates early modes to dominant velocity variation.
- **Rank 6 is always best numerically:** compactness still requires asking whether a smaller rank is almost as accurate.
- **POD fails in extrapolation:** coefficient behavior or the linear subspace does not extend to the new regime.

17.16 Optional Stretch Extensions (advanced / prize track)

Complete the required Track-3 study first. With instructor approval, choose at most one:

1. compare linear interpolation of POD coefficients in Re with the neural branch under identical modes;
2. add a matched-storage benchmark against coarse-grid bilinear reconstruction or per-field truncated SVD;
3. define a structure-aware rank rule using wall, centerline, weak-vortex, or pressure-gradient criteria;
4. extend the dataset and examine how archive compression changes as the number of stored physical cases grows.

17.17 Track-3 Required Output Package

- `P3_results.csv`;
- POD singular-value/energy curves;
- rank/basis selection table;
- reconstruction and prediction errors;

- coefficient count and compression discussion;
- low-rank versus selected-rank figure;
- one physical feature whose preservation controls rank selection;
- a bounded conclusion on the reduced representation.

17.18 Frequent Track-3 Mistakes

- computing modes with blind test fields;
- selecting rank only from cumulative energy;
- confusing output-dimension reduction with total storage compression;
- concatenating u, v, and p without scaling;
- comparing separate and shared bases without reporting their different coefficient counts;
- calling a scalar-parameter POD branch a general operator learner;
- hiding low-rank failure by plotting only streamlines;
- attributing all ROM error to POD without separating coefficient-prediction error.

18 Track 4: Uncertainty and Data Sufficiency

18.1 Why this track matters

A neural-network prediction is not a single unquestionable number. The result can change when the network is initialized differently, when the optimizer follows a different path, or when the physical training cases change. Track 4 teaches students to measure one source of variability at a time and to avoid calling every spread "uncertainty."

Notebook: `P4_Uncertainty_Study.ipynb`. The model architecture and blind cases remain fixed. Variant 4A changes training seed only. Variant 4B changes the number and placement of development Reynolds-number cases only.

Required deliverable

Starting files: `P4_Uncertainty_Study.ipynb`, `w4utils.py`, `w5_common.py`, and `cavity_data.npz`.
Typical experimental workload: 7–10 hours; report preparation is additional.
Student edits: one approved variant, a small fixed seed/case list, and one predeclared uncertainty or data-sufficiency criterion.

18.2 Three different ideas that should not be confused

18.2.1 Training variability

Even with the same data and architecture, neural training is not perfectly deterministic. Different random seeds change initial weights and sometimes the order of mini-batches. The resulting spread measures sensitivity to the training procedure.

18.2.2 Data uncertainty/noise

If the labels come from a stochastic simulation, repeated physical cases may produce different labels. That is not the same as training variability. Track 5 studies label variability; Track 4A keeps the data fixed.

18.2.3 Model-form and reference-solver error

All ensemble members can agree and still be wrong because they share the same architecture and the same imperfect labels. A small ensemble spread does not prove physical correctness.

Conditional uncertainty statement

Variant 4A estimates variability *conditional on the fixed dataset, architecture, preprocessing, and training rule.* It does not estimate total uncertainty in the CFD solution or in real physics.

18.3 Variant 4A: training-seed uncertainty

Research question for Variant 4A

How sensitive are blind cavity predictions to neural-network initialization, and does ensemble disagreement identify the locations where the prediction error is large?

Train the same model with seeds $s_1, \ldots, s_K$. Let $\widehat{q}^{(k)}(x, y)$ be the prediction from ensemble member k. The ensemble mean is

$$\overline{\widehat{q}}(x, y) = \frac{1}{K} \sum_{k=1}^{K} \widehat{q}^{(k)}(x, y),$$

and the pointwise ensemble standard deviation is

$$s_q(x, y) = \sqrt{\frac{1}{K-1} \sum_{k=1}^{K} \left(\widehat{q}^{(k)} - \overline{\widehat{q}} \right)^2}.$$

The standard deviation is an empirical disagreement measure. It is not automatically a calibrated confidence interval.

The student should compare $s_q(x, y)$ with the actual pointwise blind error

$$e_q(x, y) = \left| \overline{\widehat{q}}(x, y) - q_{\text{ref}}(x, y) \right|.$$

A useful uncertainty indicator should tend to be larger where the error is larger, but perfect correlation is not expected.

18.4 Spread-error correlation

The notebook reports a correlation between pointwise ensemble spread and actual error. Define flattened arrays $\mathbf{s}$ and $\mathbf{e}$ over the grid. Pearson correlation is

$$\rho_{s,e} = \frac{\operatorname{cov}(\mathbf{s}, \mathbf{e})}{\operatorname{std}(\mathbf{s}) \operatorname{std}(\mathbf{e})}.$$

A positive value indicates that disagreement tends to increase with error. A value near zero means spread is not locating error reliably. Correlation alone is not calibration: a spread can correlate with error but have the wrong magnitude.

What the student does:

1. Freeze one architecture, split, scaler procedure, epoch/stopping rule, and list of seeds.
2. Train every seed independently.
3. Report mean, standard deviation, best, and worst blind metrics.
4. Plot ensemble mean and pointwise spread.
5. Compare spread with actual pointwise error.
6. Identify at least one region where spread is misleadingly low or high.

18.5 Variant 4B: physical-case sufficiency

Research question for Variant 4B

How many complete Reynolds-number cases are required before the surrogate preserves both global accuracy and selected local flow features on the same blind cases?

The independent unit is a complete physical Reynolds-number case. The notebook uses nested development sets. For example, a label such as `4_dev_cases` means four permitted development cases total; one may be

reserved for validation, leaving three cases for gradient updates during architecture selection. The output table reports both `n_dev_cases` and `n_train_cases`.

Data sufficiency depends on both count and coverage. Four cases spread across the physical range may be more useful than five cases clustered at low Re. Therefore, report the actual Reynolds numbers, not only the count.

What the student does:

1. Define nested development-case sets before training.
2. Keep architecture, seed, optimizer, and blind cases fixed.
3. Train one model for each case set.
4. Plot blind error versus number of development cases.
5. Compare centerlines and at least one local feature.
6. Identify the smallest case set that meets a predeclared adequacy criterion.
7. Identify a case set that preserves global flow but loses a local feature.

18.6 Adequacy criteria

A possible data-sufficiency rule is:

- global E_{uv} below 5%;
- pressure error below a stated threshold;
- wall or centerline metric within 10% of the result obtained with the largest development set;
- correct recirculation sign/topology.

The criterion must be declared before reviewing all blind results.

18.7 Step-by-step notebook workflow

1. Audit the fixed dataset and blind cases.
2. Select Variant 4A or 4B.
3. Freeze all factors except seed (4A) or development-case set (4B).
4. Train every run and save every prediction; do not keep only the best run.
5. Build a summary table.
6. Produce uncertainty/spread evidence (4A) or error-versus-coverage evidence (4B).
7. Perform a local physical check.
8. State precisely which uncertainty or sufficiency question was answered.

18.8 Required evidence for Variant 4A

- one row per seed with blind metrics;
- mean, standard deviation, best, and worst metrics;
- ensemble-mean field and uncertainty/spread map;
- centerline mean with an ensemble band;
- spread-error correlation or an equivalent diagnostic;
- one location where spread fails to flag a large error;
- statement of uncertainty sources not included.

18.9 Required evidence for Variant 4B

- exact Reynolds numbers in every development set;
- `n_dev_cases` and `n_train_cases`;
- blind error versus case count/coverage;
- centerline or field comparison for smallest and largest sets;
- a local feature that stabilizes later than the global error;
- smallest scientifically adequate development set;
- statement explaining why grid points are not counted as independent cases.

18.10 How to interpret common outcomes

- **Small seed spread and large common error:** the model is consistently biased; seed uncertainty misses the failure.
- **Large spread near a high-gradient region:** optimization is sensitive where the mapping is difficult, but this is not yet a calibrated probability.
- **Error decreases rapidly then plateaus with more cases:** additional cases provide diminishing returns within the tested range.
- **Global error stabilizes before wall/corner behavior:** local physics needs more case coverage than the dominant field structure.
- **A sparse but well-spaced set beats a larger clustered set:** physical coverage matters as much as sample count.

Required failure test

For 4A, identify a location where ensemble spread is small but actual error is large, or vice versa. For 4B, identify the smallest case set that preserves the global vortex but loses a wall, pressure, or secondary-flow feature.

18.11 Optional Stretch Extensions (advanced / prize track)

Complete the required study first. With instructor approval, choose at most one:

1. test empirical calibration by comparing spread thresholds with actual pointwise error coverage;
2. combine training seed and development-case count in a small two-factor experiment;
3. use ensemble disagreement as an acquisition score and compare one selected additional Reynolds case with one random additional case.

18.12 Track-4 Required Output Package

- `P4_seed_results.csv` or `P4_data_size_results.csv`;
- all run-level results, not only the best run;
- ensemble spread/error evidence or error-versus-case-count evidence;
- one centerline band or multi-run comparison;
- one physical feature whose stability changes;
- a statement of what was and was not measured.

18.13 Frequent Track-4 Mistakes

- calling one training run an uncertainty study;
- changing seeds, architecture, and epochs simultaneously;
- counting grid points as independent physical cases;
- presenting ensemble standard deviation as a calibrated confidence interval without testing;
- interpreting small spread as proof of physical correctness;
- choosing the best seed and discarding the others;
- discussing case count without reporting physical coverage.

19 Track 5: Rarefied/Kinetic Cavity AI (Advanced)

19.1 Purpose and scope

Notebook: `P5_Rarefied_Cavity.ipynb`; helper: `mini_dsmc.py`. This track introduces machine learning with stochastic kinetic-style simulation labels. The scientific question is not only whether a neural network fits a field. The student must also ask how much of the observed error comes from neural approximation, stochastic label variation, finite sampling time, and out-of-range physics.

The supplied solver is a CPU-friendly teaching model. It includes particle initialization, ballistic movement, diffuse wall reflection, local stochastic collisions, and time-averaged cell moments. It is useful for learning workflow and uncertainty, but it is not a validation-quality production DSMC solver.

Required deliverable

Starting files: `P5_Rarefied_Cavity.ipynb`, `mini_dsmc.py`, `w5_common.py`, and `w4utils.py`.
Typical experimental workload: 9–14 hours; instructor approval required; report preparation is additional.
Student edits: one approved variant, bounded Kn, wall-speed, and seed lists, one case-wise split, and one stated noise/physical criterion.

19.2 Rarefaction and the Knudsen number

The Knudsen number is

$$Kn = \frac{\lambda}{L},$$

where λ is the molecular mean free path and L is the cavity length. As Kn increases, molecular non-equilibrium and wall interaction effects become more important. A surrogate trained at one range of Kn may not remain valid in another range.

Because Kn can span orders of magnitude, the network input uses

$$\kappa = \log_{10} Kn$$

rather than raw Kn. Equal steps in κ correspond to multiplicative changes in rarefaction and give the network better numerical resolution across regimes.

19.3 What is a stochastic label?

For a deterministic CFD solver, rerunning the same case with the same settings ideally gives the same field. In a Monte Carlo particle method, different random seeds produce different particle histories. After finite averaging, the estimated macroscopic fields differ slightly even when the physical condition is identical.

Let $q^{(s)}(x, y)$ be the field estimated with seed s. A simple error model is

$$q^{(s)} = q_{\text{ideal}} + b_{\text{num}} + \varepsilon^{(s)}_{\text{sample}},$$

where

- q_{ideal} is the ideal physical quantity represented by the model;
- b_{num} is shared systematic bias from grid, time step, collision model, wall model, and finite-time effects;
- $\varepsilon^{(s)}_{\text{sample}}$ is seed-dependent statistical sampling error.

Averaging independent seeds reduces $\varepsilon_{\text{sample}}$ but does not remove the shared bias b_{num}.

19.4 Simulation case, point sample, and split

One complete case is identified by

$$(Kn, U_{\text{wall}}, \text{seed}).$$

Every cell in that simulation shares the same seed history, numerical settings, and systematic biases. Therefore, grid points from one case must remain together. The case label is assigned before fields are flattened into coordinate samples.

Important boundary

A random point split would put cells from the same stochastic realization into training and testing. The model could exploit shared spatial patterns and noise, producing an unrealistically optimistic score.

19.5 Inputs and outputs of the Track-5 surrogate

The coordinate DNN input is

$$\mathbf{z} = (\log_{10} Kn,\, U_{\text{wall}},\, x,\, y).$$

The predicted outputs are

$$\widehat{\mathbf{y}} = \left(\frac{\hat{u}}{U_{\text{wall}}}, \frac{\hat{v}}{U_{\text{wall}}}, \frac{\hat{T}}{T_{\text{wall}}} \right).$$

The normalizations make cases with different wall speed more comparable. In the teaching solver, $T_{\text{wall}} = 1$ in dimensionless units, so the stored temperature is already normalized.

19.6 How the pooled temperature estimator works

A biased approach would compute a temperature from a very small particle population in every snapshot and then average those temperatures. The revised solver instead accumulates pooled moments over the entire sampling window:

$$N_{\text{tot}} = \sum_t N_t, \qquad \mathbf{S}_1 = \sum_t \sum_{j=1}^{N_t} \mathbf{c}_j, \qquad S_2 = \sum_t \sum_{j=1}^{N_t} |\mathbf{c}_j|^2.$$

The pooled mean velocity is

$$\overline{\mathbf{c}} = \frac{\mathbf{S}_1}{N_{\text{tot}}},$$

and the thermal variance is computed from $S_2/N_{\text{tot}} - |\overline{\mathbf{c}}|^2$ with a finite-sample correction. Pooling greatly reduces the small-sample temperature bias, but it does not eliminate transient or model-form bias.

19.7 Sampling window and transient bias

The default run uses 3200 steps and begins sampling after step 2400. This later window reduces contamination from the initial transient. However, it does not prove that every case has reached an asymptotic steady state. If relaxation time depends on Kn, residual transient bias may vary with Kn and affect Variant 5B.

The report must therefore record:

- grid size and particles per cell;
- time step, total steps, and sampling start;

- sampling stride and number of field samples;
- seed and physical case;
- temperature estimator;
- simplified collision and wall models;
- whether the label is a single seed or seed average.

19.8 Variant 5A: noise-aware single-seed versus seed-averaged labels

Research question for Variant 5A

Does training on an average of two independent stochastic labels improve prediction against a third independent realization, compared with training on one seed only?

For each physical condition (Kn, U_{wall}), the supplied design uses:

- seed 11: single-seed training label;
- seeds 11 and 22: two-seed averaged training label;
- seed 33: independent test realization.

The two models are trained with the same architecture and development conditions. They are compared against the same independent seed-33 cases.

The averaged label is

$$\overline{q}_{12}(x, y) = \frac{q^{(11)}(x, y) + q^{(22)}(x, y)}{2}.$$

If seed errors are independent with variance σ^2, the variance of the average is approximately $\sigma^2/2$. The reduction is statistical; shared bias remains.

Required analysis:

1. Quantify the seed-11/seed-22 label gap before training.
2. Train one model on seed 11 and one on the two-seed average.
3. Test both against complete seed-33 cases.
4. Compare velocity and temperature errors.
5. Determine whether improvement is larger than the seed-to-seed label variability.
6. Identify a region where neural smoothing hides stochastic cell-level variation.

19.9 Variant 5B: generalization across Knudsen number

Research question for Variant 5B

Can a surrogate trained on lower-Kn cavity cases generalize to an unseen higher-Kn case, and which physical or data-quality feature indicates that the model has left its reliable regime?

The default split averages the available seeds at each physical condition and uses:

$$Kn \in \{0.05, 0.10\} \quad \text{for training,}$$

$$Kn = 0.20 \quad \text{for validation,} \qquad Kn = 0.40 \quad \text{for blind testing.}$$

The full (Kn, U_{wall}) cases remain intact. The claim is about transfer across rarefaction conditions, not random grid points.

The student should analyze whether failure is caused primarily by:

- out-of-range Kn physics;
- insufficient representation capacity;
- stochastic label variability;
- finite-time/transient differences between Kn cases;
- a physical check that was not enforced by the training loss.

19.10 Physical checks for Track 5

At least two are required; prize-track work should use three or more.

- **Temperature positivity:** $\hat{T} > 0$ everywhere.
- **Lid-driven sign:** mean predicted u/U_{wall} near the top should be positive.
- **Return-flow sign:** the lower cavity should contain negative u/U_{wall} consistent with recirculation.
- **Centerline shape:** compare the vertical u profile with the held-out simulation.
- **Seed variability:** compare surrogate residuals with differences among repeated seeds.
- **Regime trend:** determine whether changes with Kn are smooth and physically interpretable within the teaching model.

19.11 Step-by-step notebook workflow

1. Choose Variant 5A or 5B and freeze the case list.
2. Generate or load every complete simulation case.
3. Audit metadata and field shapes.
4. Assign case labels before flattening fields.
5. Build development, validation, and blind case groups.

6. Fit normalization statistics on development cases only.
7. Train the fixed coordinate DNN.
8. Evaluate complete held-out cases.
9. Compare numerical errors with seed variability and physical checks.
10. State which error sources can and cannot be separated by the experiment.

Case rows used by the DNN

```
# One point row from one complete physical case:
X_row = [np.log10(Kn), Uwall, x, y]
Y_row = [u/Uwall, v/Uwall, T/Twall]
```

The point rows are created only after each full case has been assigned to development, validation, or blind test.

19.12 Required evidence for Variant 5A

- complete table of physical conditions and seeds;
- seed-to-seed label-gap metric;
- single-seed versus two-seed-averaged training comparison;
- evaluation against independent seed 33;
- velocity and temperature field/centerline evidence;
- at least two physical checks;
- statement separating random variance from shared bias.

19.13 Required evidence for Variant 5B

- exact train, validation, and blind Kn values;
- complete case-wise split table;
- blind velocity and temperature errors at $Kn = 0.40$;
- centerline and recirculation evidence;
- at least two physical checks;
- discussion of possible Kn-dependent transient bias;
- bounded statement of the reliable rarefaction range.

19.14 How to interpret common outcomes

- **Averaged labels improve independent-seed error:** random label noise was limiting the single-seed surrogate.

- **Averaging changes little:** model bias or shared simulation bias may dominate.
- **Smooth neural field has smaller cellwise noise but wrong centerline:** denoising appearance is not physical accuracy.
- **High-Kn velocity error increases while temperature remains positive:** positivity is necessary but too weak to establish regime validity.
- **All seeds agree but the sampling window is early:** seed agreement does not remove common transient bias.

Required failure test

A smooth predicted field is not evidence that stochastic simulation labels are converged. The report must identify at least one condition where surrogate error, seed variability, or out-of-range behavior prevents a trustworthy claim.

19.15 Optional Stretch Extensions (advanced / prize track)

Complete the required study first. With instructor approval, choose at most one:

1. compare early and late sampling windows to separate transient bias from seed variance;
2. estimate cellwise label variance from repeated seeds and use inverse-variance weighting;
3. build a compact shared-support representation of kinetic fields and compare fidelity versus storage against grid/POD baselines;
4. add one additional Kn case selected from an error or ensemble-disagreement criterion and test whether it improves transfer.

19.16 Track-5 Required Output Package

- `P5_results.csv`;
- cached simulation archive used in the final run;
- complete $(Kn, U_{\text{wall}}, \text{seed})$ table;
- seed-variability or unseen-Kn comparison;
- at least two physical checks;
- data-quality and solver-limitation paragraph;
- explicit separation of label noise, surrogate error, transient bias, and regime extrapolation where possible.

19.17 Frequent Track-5 Mistakes

- randomly splitting cells from the same stochastic case;
- treating different seeds as different physics;
- claiming the mini solver is validation-quality DSMC;

- concluding that a smoother field is more physically accurate;
- comparing a model with the same seed used for its labels;
- assuming seed averaging removes systematic bias;
- changing Kn, grid, particles, and sampling time together without identifying confounding factors;
- omitting the sampling window and metadata;
- using raw Kn across orders of magnitude without justification.

20 Track 6: GPU-Native Fokker–Planck Neural Closure for the Cavity (Advanced Research Track)

20.1 Purpose, audience, and research connection

Notebook: `P6_FP_Cavity_Closure.ipynb`. This is an instructor-assigned research track intended for a student who will present the associated Fokker–Planck/AI work and may continue it after the course. It mirrors the workflow of the instructor's GPU-native cubic-Fokker–Planck surrogate study at a deliberately reduced educational scale.

The associated research question is more demanding than the coordinate-surrogate questions in Tracks 1–5. The network does not predict the final cavity field directly. It predicts the local coefficients of a kinetic closure, and those coefficients are inserted into a stochastic particle solver for thousands of time steps. A model that looks accurate in an offline coefficient table may still fail after closed-loop deployment. For that reason, substantially more code is supplied and the student is not asked to rewrite the particle solver or the exact closure equations.

Required deliverable

Starting files: `P6_FP_Cavity_Closure.ipynb` and the complete `Track6_FP_Cavity` folder.
Hardware: a Google Colab GPU or approved Unity GPU node; a CPU-only final run is not required. If CuPy/CUDA is unavailable, select `Runtime -> Change runtime type -> GPU`, restart, and run the installation cell.
Typical workload: 12–18 hours for the reduced experiment, plus report and presentation preparation. Production-resolution continuation requires substantially more computation.
Student edits: bounded case lists, the q-weight, the final-run budget, and interpretation. The supplied FP equations, exact 9×9 closure, feature ordering, target ordering, and GPU forward pass are not to be rewritten without approval.

20.2 Why a Fokker–Planck model is used

In a dilute rarefied gas, the velocity distribution function evolves under transport and molecular collisions. Direct simulation Monte Carlo represents collisions by discrete random binary events. A particle Fokker–Planck model replaces the collision operator by a continuous drift–diffusion process in velocity space. In simplified notation,

$$\frac{\partial F}{\partial t} + V_i \frac{\partial F}{\partial x_i} = -\frac{\partial}{\partial V_i}(A_i F) + \frac{1}{2}\frac{\partial^2}{\partial V_i \partial V_j}(D_{ij} F),$$

where $F(\mathbf{V}, \mathbf{x}, t)$ is the velocity distribution, A_i is the drift, and D_{ij} is the diffusion tensor.

The supplied reference uses a cubic drift model. Its local stress and heat-flux relaxation depend on nine state-dependent coefficients: six symmetric coefficients C_{ij} and three coefficients Γ_i. These coefficients are not arbitrary trainable constants. In the exact solver, every cell and time step requires high-order particle moments, assembly of a dense local 9×9 system, and a direct solve. This repeated closure calculation is the target of the neural surrogate.

20.3 What the neural network learns

The neural network uses only low-order quantities already available in the inexpensive online path. Its input is

$$\mathbf{x}_{FP} = (\rho, T, U_x, U_y, U_z, \Pi_{xx}, \Pi_{xy}, \Pi_{xz}, \Pi_{yy}, \Pi_{yz}, \Pi_{zz}, q_x, q_y, q_z, DM2, \nu) \in \mathbb{R}^{16}.$$

The exact target is

$$\mathbf{y}_{FP} = (C_{xx}, C_{xy}, C_{xz}, C_{yy}, C_{yz}, C_{zz}, \Gamma_x, \Gamma_y, \Gamma_z) \in \mathbb{R}^9.$$

The project therefore learns

$$\widehat{\mathbf{y}}_{FP} = f_\theta(\mathbf{x}_{FP}).$$

The low-order feature vector is not guaranteed to identify every possible non-equilibrium velocity distribution uniquely. Two distributions can share the same low-order moments while differing in high-order structure. The model must therefore be described as a regression on the distribution manifold represented by the training simulations, not as a universal kinetic closure.

20.4 How exact training labels are generated

The script `generate_fp_cavity_dataset.py` runs the supplied exact particle-FP solver. At each selected post-transient sampling step it:

1. advances particles and applies diffuse wall reflection;
2. computes cell density, temperature, velocity, pressure-tensor moments, heat flux, and the high-order moments required by the exact cubic-FP closure;
3. assembles and solves the local 9×9 system;
4. saves the 16 low-order features and the 9 exact closure coefficients for every supported cell;
5. records the complete physical condition, sampling step, cell index, particle support, and non-equilibrium indicators.

Each row is therefore a local cell state, but the train/validation/blind assignment is made by complete cavity condition before rows are pooled. Randomly splitting neighboring cells from the same simulation would create optimistic leakage and is not permitted as evidence of physical transfer.

20.5 Reduced educational budget versus research budget

The default notebook uses grids of roughly 16^2–24^2 cells, tens to one hundred particles per cell, and shortened transients so that the workflow can be completed on a free GPU. The instructor manuscript uses far larger particle populations, longer sampling windows, additional conditions, and stronger convergence diagnostics.

The class result can establish that the end-to-end workflow functions and can support a bounded comparative claim. It cannot, by itself, replace production grid/particle/time convergence. Any conference or journal continuation must increase the physical budget, repeat seeds, audit transient convergence, and preserve complete-condition testing.

20.6 Controlled comparison: uniform loss versus q-weighted loss

The baseline network minimizes a uniform mean-squared error in standardized output space,

$$\mathcal{L}_{\text{uniform}} = \frac{1}{N} \sum_{n=1}^{N} \sum_{j=1}^{9} \left(\widehat{y}_{n,j}^{(s)} - y_{n,j}^{(s)} \right)^2.$$

The modification uses

$$\mathcal{L}_q = \frac{1}{N} \sum_{n=1}^{N} \sum_{j=1}^{9} w_j \left(\widehat{y}_{n,j}^{(s)} - y_{n,j}^{(s)} \right)^2,$$

where the in-plane heat-flux coefficients in the Γ block receive larger weights. Superscript (s) denotes standardized coefficients.

The q-weight is not automatically beneficial. It may reduce Γ error while increasing stress-block error or closed-loop field error. The required experiment therefore keeps architecture, optimizer, seed, training data, validation condition, scaling, and stopping rule fixed. Only the output-loss weights change.

20.7 Recommended complete-condition protocol

The guided notebook uses the following default design at nominal $Kn \approx 0.15$:

$$U_{\text{lid}} \in \{100, 200, 400, 600\} \text{ m/s} \quad \text{for training,}$$

$$U_{\text{lid}} = 500 \text{ m/s} \quad \text{for validation,} \qquad U_{\text{lid}} = 800 \text{ m/s} \quad \text{for blind testing.}$$

The blind 800 m/s data must not be generated or opened until the loss choice, q-weight, architecture, seed, and selection rule are frozen.

A recommended selection rule is: choose the q-weighted model only if it lowers validation Γ-block relative L_2 error and does not increase C-block error by more than 20% relative to the uniform baseline. If this criterion fails, retain the uniform model and report the negative result.

20.8 Network architecture and native parameter export

The supplied model is

$$16 \to 128 \to 128 \to 128 \to 128 \to 9$$

with ReLU hidden activations and a linear output layer. The exact width may be adjusted only with approval; both loss variants must use the same architecture.

Keras is used for training, but the particle solver does not call `model.predict()` inside its time loop. The script exports

$$W_1, \ldots, W_5, \quad b_1, \ldots, b_5, \quad \boldsymbol{\mu}_x, \boldsymbol{s}_x, \quad \boldsymbol{\mu}_y, \boldsymbol{s}_y$$

to one `.npz` file. The CuPy solver loads these arrays once and evaluates the forward pass directly on the GPU:

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{x}^{(s)} W_1 + b_1), \ldots, \widehat{\mathbf{y}}^{(s)} = \mathbf{h}_4 W_5 + b_5.$$

The output is then unscaled and inserted into the particle velocity update. Avoiding repeated CPU–GPU transfers is part of the scientific/computational design, not merely a coding convenience.

Implementation note: the legacy scalar `coeffs['C']`

The supplied solver stores the nine learned outputs in `coeffs['A']` (the six C_{ij} coefficients) and `coeffs['B']` (the three Γ_i coefficients). A separate legacy scalar array, `coeffs['C']`, appears in the particle-update expression. In the supplied reference implementation this scalar remains zero in both the exact 9×9 path and the ML path. Consequently, the explicit `coeffs['C'].fill(0)` line in native inference preserves exact/ML symmetry; it is not a tenth learned output and is not an ML-only simplification.

20.9 Offline coefficient validation versus closed-loop validation

Track 6 has two distinct levels of evidence.

Offline coefficient validation evaluates exact and neural C_{ij}, Γ_i values on complete held-out states. Required metrics include:

- relative L_2, RMSE, and MAE for all nine outputs;
- separate errors for the six-component C block and three-component Γ block;
- Γ error on the highest-q_{norm} subset, where heat-flux relaxation is most demanding.

Closed-loop validation inserts the selected model into a complete blind cavity simulation. This is stronger because coefficient errors affect subsequent particle states and may accumulate. The model must be compared with the exact-FP run using the same blind condition, numerical budget, seed policy, transient, and averaging window.

20.10 Required closed-loop evidence

At minimum, compare:

- speed, temperature, normalized density, and normalized pressure fields;
- vertical u/U_{lid} and horizontal v/U_{lid} centerlines;
- exact-FP and ML-FP runtime for the same reduced configuration;
- the time-averaged closure coefficient fields;
- at least two high-order diagnostics such as q_{norm}, $R_{ij,\text{norm}}$, $\Delta_{4,\text{norm}}$, or $DM6_{\text{norm}}$.

High-order fields are often more sensitive than speed or temperature. Agreement of smooth macroscopic contours does not prove that the evolved kinetic state retains higher-moment fidelity.

20.11 Physical and stability checks

At least three checks are required:

- density and temperature remain positive;
- no non-finite particle, field, or coefficient values are produced;
- the upper cavity has positive mean u/U_{lid} in the lid direction;
- a negative-u return-flow region exists in the lower cavity;
- centerline shape and recirculation are consistent with the exact-FP run;
- high-order diagnostic errors are reported, not hidden behind macroscopic agreement;
- the predicted coefficients remain finite and the rollout completes without coefficient clipping unless clipping was explicitly approved and disclosed.

The cubic-FP reference does not provide a universal H-theorem, and this project does not claim one for the neural surrogate. An entropy proxy may be used only as an optional like-for-like fidelity diagnostic.

20.12 Runtime interpretation

Report the measured online speedup

$$S = \frac{t_{\text{exact FP}}}{t_{\text{ML FP}}}$$

for the stated grid, particle count, GPU, and number of steps. The acceleration is produced by *two linked substitutions*: the exact FULL high-order moment pass is replaced by the LITE low-order pass, and the exact closure assembly/direct 9×9 solve is replaced by the GPU-native DNN forward pass. Therefore, S is the speedup of the complete deployed closure pipeline, not the isolated speed of neural inference. Component timing should distinguish FULL moments, LITE moments, direct solve, and DNN inference whenever possible. Do not call this a universal theoretical maximum. The remaining particle movement, wall treatment, low-order moments, stochastic update, and Python/GPU overhead bound the total speedup.

For a research continuation, also report offline training and data-generation cost and estimate the break-even number of repeated target simulations. The class project may report the online comparison only, provided the omitted offline cost is stated explicitly.

20.13 Step-by-step notebook workflow

1. Run the environment and file checks on a GPU runtime.
2. Freeze training, validation, and blind lid speeds.
3. Generate exact-FP training and validation datasets.
4. Audit shapes, feature names, target names, non-finite rows, and q_{norm} coverage.
5. Train the uniform-loss baseline.
6. Train the q-weighted model with the same architecture, seed, optimizer, and stopping rule.
7. Compare complete-condition validation C and Γ errors.
8. Write and save the model-selection decision before opening blind data.
9. Generate the blind 800 m/s coefficient dataset and evaluate both models.
10. Deploy the selected model inside the complete blind cavity simulation.
11. Generate field, centerline, high-order, physical-check, and runtime evidence.
12. Identify a failure, tradeoff, or unsupported claim.

20.14 What is supplied and what the student changes

Supplied and fixed:

- particle transport, diffuse wall reflection, moment definitions, exact cubic-FP closure, and stochastic velocity update;
- the 16-feature and 9-target ordering;
- the Keras-to-CuPy parameter export format;
- data-generation, evaluation, closed-loop, plotting, and physical-check functions;
- a complete fast-mode configuration for debugging.

Student decisions:

- fast versus final numerical budget;
- one predeclared q-weight and selection tolerance;
- validation interpretation and selected model;
- optional publication-path extension after the required study;
- bounded scientific conclusion and limitation statement.

20.15 Required failure or limitation analysis

At least one limitation must be quantified. Examples include:

- q-weighting reduces Γ error but causes unacceptable C or field degradation;
- offline coefficient accuracy does not translate to closed-loop high-order fidelity;
- the 800 m/s blind state lies too far from the training manifold;
- reduced particle counts make coefficient labels or high-order diagnostics noisy;
- the educational transient is too short for a publication-level steady-state claim;
- the low-order input map is not universally identifiable for arbitrary distributions;
- measured speedup is small because non-closure work dominates the reduced configuration.

Required failure test

The project is not successful merely because the ML solver completes. The student must show whether the trained closure is accurate, stable, physically defensible, and computationally useful for the frozen blind condition. A negative result supported by complete evidence is acceptable.

20.16 Optional prize/research extension

Complete the required speed-transfer study first. With instructor approval, choose one:

1. run an unseen rarefaction test at nominal $Kn \approx 0.5$ without retraining and define the breakdown of the original $Kn \approx 0.15$ training manifold;
2. add complete higher-Kn training cases and reserve a new untouched rarefaction condition for blind testing;
3. repeat the blind closed-loop run over multiple particle seeds and compare neural error with exact-FP sampling variability;
4. perform component-level GPU profiling and a break-even analysis including offline data generation and training;
5. compare macroscopic fidelity with higher-order or entropy-proxy fidelity under production-resolution settings.

The first option is already scaffolded in the notebook by changing the density scale. It is a stress test, not routine interpolation.

20.17 Track-6 required output package

- fully executed `P6_FP_Cavity_Closure.ipynb`;
- exact train/validation/blind case table and metadata;
- uniform and q-weighted training logs and coefficient metrics;
- saved pre-blind model-selection record;
- selected exported CuPy parameter file;
- exact-FP and ML-FP closed-loop NPZ outputs;
- field, centerline, high-order, physical-check, and runtime files;
- a bounded conclusion suitable for the associated conference presentation;
- explicit statement that reduced educational runs are not manuscript-level convergence evidence.

20.18 Frequent Track-6 mistakes

- randomly splitting cells or time samples from one condition and calling the result physical generalization;
- opening the 800 m/s blind data before freezing the q-weight and architecture;
- changing q-weight, architecture, data, and seed simultaneously;
- reporting only Keras validation loss rather than physical-unit coefficient blocks;
- assuming lower Γ error guarantees better closed-loop fields;
- comparing exact and ML runs with different numerical budgets;
- claiming a universal closure, H-theorem, or hardware-independent speedup;
- presenting reduced classroom particle counts as production validation;
- modifying the supplied closure equations without instructor approval.

Central message

Software compatibility target: Tracks 1–5 target Google Colab/Python 3.12 with Keras 3. Track 6 additionally requires an NVIDIA GPU and CuPy. The reference continuum dataset hash is recorded in `reproducibility.txt`; Track-5 cached files and Track-6 generated datasets are versioned by their metadata and complete case definitions.

21 Week-5 Checkpoint Template

Use the following headings in a one- to two-page checkpoint note.

1. Project variant and research question.
2. Files and dataset used.
3. Frozen train/validation/test design.
4. Baseline reproduction result.
5. Definition of the main variables/rank/loss weight being changed and the expected effect.
6. Modification completed so far.
7. Preliminary metric table or figure.
8. First physical check.
9. Planned failure test.
10. Current technical obstacle.
11. AI/code-assistance log.

22 One-Page Project Proposal Form

Complete this before running the main experiment.

Student/team	_____
Selected variant	_____
One-sentence research question	_____
Baseline method	_____
One primary modification	_____
Training cases	_____
Validation case(s)	_____
Blind test case(s)	_____
Primary numerical metric	_____
Physical check 1	_____
Physical check 2	_____
Predeclared failure criterion	_____
Expected limitation	_____
AI assistance planned	_____

Important boundary

If the proposal contains more than one primary modification, reduce the scope. The six-week course rewards a complete controlled study more than an unfinished collection of ideas.

23 Final Report Structure

Recommended 6–10 page report (appendices may contain extra plots or code details):

1. **Question and claim** - one paragraph stating the exact research question and the bounded claim to be tested.
2. **Physical/data setting** - define Re or Kn, the complete simulation case, fields, normalization, and reference-data limitation.
3. **Data and split** - training, validation, development, and blind cases; explain how leakage is prevented.
4. **Baseline** - architecture/method, important variables, and reproduced result.
5. **Modification** - define every symbol in the new loss, rank rule, uncertainty quantity, or data-quality comparison; explain the relevant code path.
6. **Controlled experiment** - state what changed and what was held fixed.
7. **Numerical results** - concise tables with units/normalizations and clear selection criteria.
8. **Physical validation** - at least two checks; prize-track projects should use three or more.
9. **Failure/tradeoff** - required negative or limiting evidence and the likely mechanism.
10. **Conclusion** - whether the modification is useful, for which regime and purpose, and what cannot be claimed.
11. **Reproducibility and AI-use statement** - files, seeds, versions, and verification of AI-assisted code.

24 Grading Rubric

Criterion	Weight	Evidence
Baseline reproduction	10%	Correct fixed starting point and files
Scientifically correct modification	20%	One bounded extension, implemented and explained
Controlled comparison	20%	Frozen split; fair ablation; no test tuning
Physical validation	20%	At least two relevant checks
Failure/tradeoff analysis	15%	Documented limitation with quantitative evidence
Interpretation, reproducibility, defense	15%	Clear claim, saved outputs, code understanding, AI log

25 Pre-Submission Checklist

☐ My notebook runs from a fresh runtime with the listed files.

☐ I did not alter the CFD solver unless my approved project explicitly required it.

☐ I froze model/split choices before opening blind results.

☐ My comparison changes one primary factor at a time.

☐ I report numerical and physical metrics.

☐ I include a failure or tradeoff, not only a success case.

☐ Every figure has axis labels, units/nondimensionalization, and a caption.

☐ My PDF values match the final executed notebook.

☐ I include seeds, versions, runtime, and hardware.

☐ I disclose and verify any AI-assisted code.

☐ I can explain one key code cell without external assistance.

26 Frequently Asked Questions

Do I need to create a new CFD or kinetic solver?

No. Tracks 1–4 use fixed Week-4 data, Track 5 uses the supplied teaching simulator, and Track 6 uses the supplied particle-FP reference solver plus bounded wrappers. Track-6 students train and deploy the neural closure but do not rewrite the exact closure equations.

Must my modification improve every metric?

No. A credible tradeoff or negative result is acceptable and often more instructive.

May I choose a different project?

Yes, only with written approval and an equivalent baseline, controlled comparison, two physical checks, and failure test.

Is DeepONet required?

No. Use the model appropriate to the question. Complexity is not a grading criterion by itself.

May I ask an AI tool to generate code?

You may use AI assistance under the course policy, but you must disclose it, verify it, and defend the resulting implementation. Unexplained generated code does not satisfy the project.

What if my project fails?

Report the failure quantitatively, identify the likely reason, and state the valid scope of the method. An honest, well-controlled failure can earn full credit.